

Key Difference

Type	What it is	Typical Use
ActionResult	A <i>class</i> that derives from ActionResultBase and represents a specific HTTP response (e.g., ViewResult, JsonResult).	When you always return one concrete type or a known set of ActionResult-derived types.
IActionResult	An <i>interface</i> implemented by ActionResult.	When you want flexibility to return any object that can produce an HTTP response, including your own custom classes.

Why Prefer IActionResult in Some Cases

Using the interface makes your controller action more flexible and testable.

Scenario: Multiple Return Paths

Suppose you have an endpoint that might:

- Return Ok() (HTTP 200) with a payload if everything succeeds, **or**
- Return NotFound() (HTTP 404) if the item doesn't exist.

[HttpGet("{id}")]

```
public IActionResult GetProduct(int id)
{
    var product = _repository.Find(id);

    if (product == null)
        return NotFound(); // returns a NotFoundResult

    return Ok(product);    // returns an OkObjectResult
}
```

Here, the method can return different concrete result types.

If you declared it as ActionResult, you'd be forced to return only classes derived from ActionResult (which these are), but using IActionResult signals to consumers and test code that *any* type implementing the interface is acceptable—including custom results you might create later.

Testing Benefits

When unit testing, you can mock or create your own implementation of `ActionResult` without tying tests to the full ASP.NET MVC pipeline.

This makes controller tests simpler and isolates them from framework internals.

When to Stick with `ActionResult<T>`

In ASP.NET Core 2.1+ you'll often see `ActionResult<T>`, which combines the best of both worlds:

- Lets you return a strongly typed object (T) **or**
- Any `ActionResult` (like `NotFound()`).

Example:

```
[HttpGet("{id}")]
```

```
public ActionResult<Product> GetProduct(int id)
```

```
{
```

```
    var product = _repository.Find(id);
```

```
    if (product == null)
```

```
        return NotFound();
```

```
    return product; // automatically wrapped in OkObjectResult
```

```
}
```

This is great when your default success path returns a `Product`, but you might need to send an error response.

- **Use `ActionResult`** when you need maximum flexibility or plan to inject/test with custom result implementations.
- **Use `ActionResult` or `ActionResult<T>`** when your action's return types are well defined and you want stronger compile-time checks.